JDBC: Java Database Connectivity
- It is API that enables Java Appl^n to interact with database such as
  - MySQL
  - Oracle
  - SQL Server & others

Using JDBC, a java program can',
- Establish a connection with a DB.
- Execute SQL queries.
- Retrive.

JDBC Provides 2 major components.

(1) JDBC API:
- It consist of classes & interfaces available in the java.sql & javax.sql packages
- Database - Independent.
- Remains same regardles of the DB being used. (MySQL, Oracle, SQL Server, etc.).

(2) JDBC Drivers:
- A Driver is a software components that enables that enables communication between a Java Appl^n & specific DB.
- DB Dependent.

Internally Its a class.

* Types of JDBC Drivers.

Type 1 - JDBC-ODBC Bridge Driver.

- Requires ODBC to be installed on client machine.
- Performance is poor & Not Recommended.

Type 2 - Native API Driver.

- Requires database-specific client libraries to be installed on client machine.
- Platform dependent.
- Cannot be used for Internet-based Appl^n
- Not suitable for distributed env.

Type 3 - Network Protocol Driver.

- Used a middleware (application server) to connect the DB.
- JDBC calls are sent to middleware, which then communicate with DB.

Adv- No DB client libraries required.
DisAdv- Requires middleware setup & slower than Type 4.

Type 4 - Thin Driver (com.mysql.cj.jdbc.Driver)

- Pure Java Driver which directly connects to the DB using DB specific protocol.
- Doesn't require anything externally.
- Faster & most efficient.
- Platform independent & widely used.

Steps Required for JDBC Appl?

1. load & register driver with DriverManager
2. Get connection with database
3. Communicate with database with the help of JDBC API.

```
public class Appl.
{
public static void main(String[]args)
{
String ss = "jdbc:mysql://localhost:3306/mydb";
```

main protocol. sub protocol. machine add. port. db name.

credentials

```
try( Connection con = DriverManager.getConnection(ss,"root","pass")
  {
```

Interface. Implementation of Interface? class. static method.

```
Statement st = con.CreateStatement();
```

Create Statement.

```
ResultSet rs = st.executeQuary("Select * from dept");
```

Result in form of Resultset.

```
while(rs.next())
{ int no = rs.getInt("deptno");
  String name = rs.getString("dname");
```

by default pointer is before first record. (it move pointer next)

```
}
catch(Exception ee)
  {
  }...
}
}
}
```

* Resultset: ← JDBC Interface
— represents tabular data.

if (rs != null) ✗
(rs.next()) ✓ } Resultset represent a cursor over db row not collection or single object.

- Its always valid object after executeQuery() even if the query returns zero rows.

* JDBC API:
- Connection / statement / Resultset.
- Enable loose coupling between appl^n & db vendors.
- JDBC defines Interfaces.
- Vendors provide implementation class.
- Java code depend on interfaces not implementations.

* Before Type 4 driver,
- We have to load the driver explicitly.

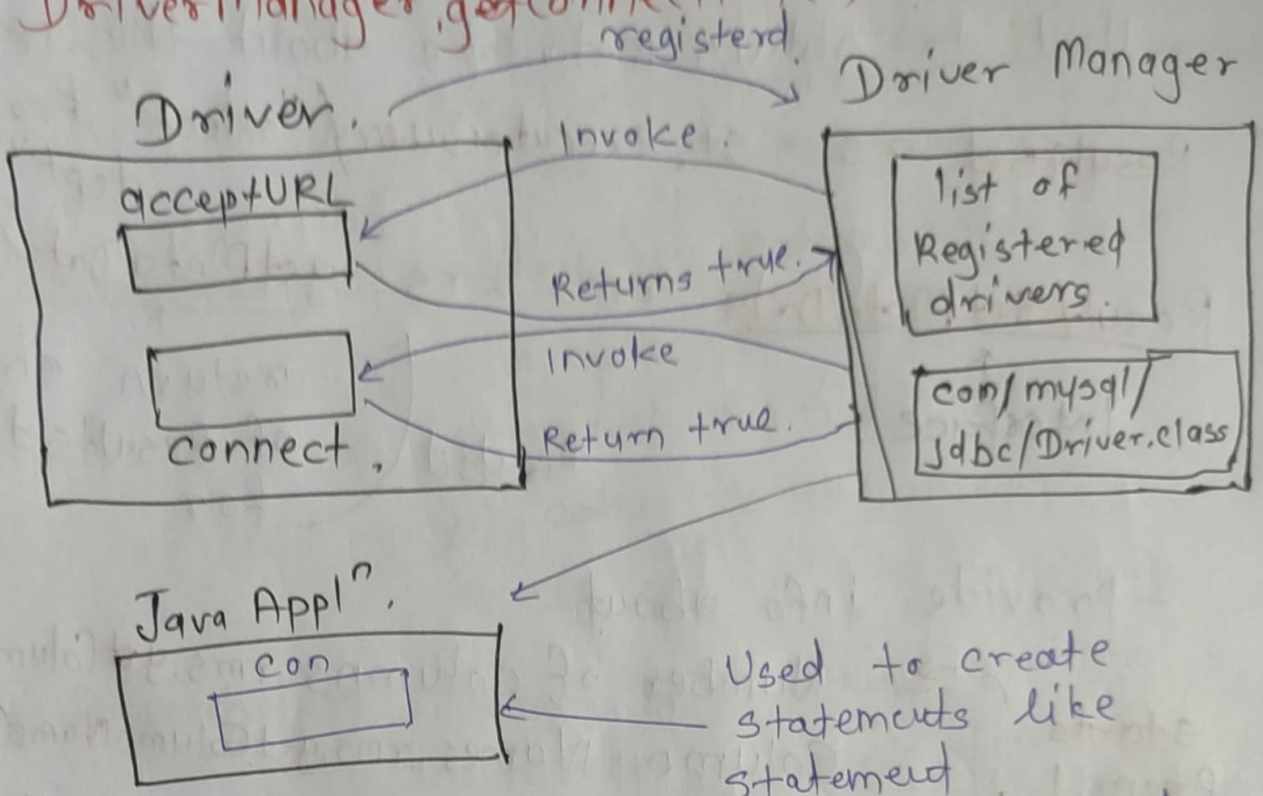
Class.forName ("com.mysql.cj.jdbc.Driver");

load jdbc driver.

It has static block. In which the logic of registration is defined (It register the driver with DriverManager.

As soon as we load the driver it with automatically gets register.

* Internal Working of DriverManager.getConnection()



① method Invocation ← method called by Applⁿ.

② Driver Discovery ← DriverManager scall, all registered JDBC driver available in the classpath.

③ Driver Selection ←

④ URL parsing ← driver extract protocol, host, port & DB name from URL.

⑤ Network Connection ← driver establish, TCP/IP socket connection with DB server.

⑥ Authentication ← driver sends yname & pass to DB for verification.

⑦ Connection Creation.

* ResultSetMetaData: ← Interface in java.sql.

- Used to obtain metadata about the resultset

```
ResultSet rs = st.executeQuery("select * from dept");
ResultSetMetaData rms = rs.getMetaData();
```

ResultSetMetaData ← Interface.

rs ← ResultSet. getMetaData() → return obj of ResultSetMetaData for

- Provide info about
  - Number of columns rms.getColumnCount()
  - Column Names rms.getColumnName(i) (starts from 1.)
  - Column datatypes
  - rms.getObject(i) ← by column Index. (i → position of column.)

* PrepareStatement.

- Sub interface of statement.
- Represent precompiled SQL statement.
- SQL Query is sent to db only once & can executes multiple times.

```
PreparedStatement pst = con.prepareStatement("select * from dept");
```

PreparedStatement ← Interface; pst ← obj

The db: - Parses the SQL - Compiles it. - Store the execution plan.

```
pst.executeQuery();
```

Adv of PrepareStatement

① Security : - Prevents SQL Injection attacks.
- User input treated strictly as data
- Not the part of SQL logic.

② Performance : - query is precompiled by db.
- Eliminate repeated parsing, optimization & compilation.
- DB catch execution plan & reuse it.

③ Reusability - Same SQL query can be executed multiple times.
- Only parameter value change.

④ Better Connection Pooling :
- They can be efficiently reused in connection pooling env.

Why Statement Is Less Efficient

- Each execution require db for.
  - Parse the SQL Query
  - Optimize it.
  - Compile it into an execution plan.
- The process is expensive, especially for repeated execution.
- No execution plan reuse.
- doesnt support parameters.

* Parameterized SQL (? placeholders).

update dept set loc=? where deptno=?

parameter placeholder

- Actual values are supplied later using setters.

```
PrepareStatement pst = con.prepareStatement("update dept set loc=?");
```

refers to 1st placeholder.

```
pst.setString(1, "delhi");
```

setter — replace 1st question mark with delhi.

bind string value to the specific placeholder position.

```
int mod = pst.executeUpdate();
```

executes DML statements
- INSERT
- UPDATE
- DELETE.

- Return int representing the number of rows affected.

(mod > 0) → One more record modified.

(mod == 0) → No records affected.

* Stored Procedure in MySql.

Input parameter.

```
DELIMETER //  ← changes terminator
Create procedure mypro(IN no INT,
                       OUT name VARCHAR(20))
BEGIN
    SELECT dname INTO name
    FROM dept
    WHERE deptno = no;
END //
DELIMITER;
```

define store procedure

O/p used to return value from

assign query result to an OUT parameter.

It store precompiled SQL program. stored on server.

* Callable Statement

- Sub interface of PreparedStatement.
- Used to call stored procedure in DB.
- Supports IN, OUT & INOUT parameters (only parameters change).

prepare call to stored procedure.

OUT parameter

IN parameter

```
CallableStatement cs = con.prepareCall("{call mypro(?,?)}");
```

index value

```
cs.setInt(1,3);   ← replace 1st parameter with 3.
cst.execute();    ← executes stored procedure (return values via OUT parameters) (No ResultSet is returned).
name = cs.getString(2);
```

retrive value of an OUT parameter.

return the value from 2nd (?).

* Bydefault a Resultset is - Read-only
- Forward-only.

Record cannot be modified while traversing the Resultset.

Can traverse only in one direction using next() method.

We can make it

* Updatable Resultset.
  - It allows - Modifying data (record) while iterating through resultset.
  - Operations such as - updateXXX()
    - updateRow()
    - updateRow()

* Scrollable Resultset:
  - Allows - Moving cursor both direction.
    - Random access to row.
  - common Navigation Methods.
    - next()
    - previous()
    - first()
    - last()
    - absolute (int row)
    - relative (int rows).

* ResultSet.TYPE_SCROLL_INSENSITIVE
  ↳ make it scrollable.
- while traversing the ~~db~~ resultset if db changed externally we wont come to know.
- The data view Remains static.

* ResultSet.TYPE_SROLL_SENSITIVE
  ↳ Resultset is scrollable.
- If DB data is modified externally, changes may be reflected while traversing the Resultset.

It depend on driven (mysql doesn't provi sensiti

In both - Cursor can move both forward & backward.

* ResultSet.CONCUR_UPDATABLE
  ↳ make Resultset updatable.
- Allow modification of DB directly th the Resultset.

Create scrollable & Updatable Resultset

```
Statement st = con.createStatement(
        ResultSet.TYPE_SCROLL_INSENS
        ResultSet.CONCUR_UPDATABLE
);
```

↗ concurrency mode.

rs.absolute(int row) ← move cursor directly to a specific row number.

rs.previous() ← move previous.

rs.updateString(String column, String value) ← update column value in the curreed row (in memory).

rs.updateRow() ↑ commits the update row data to the DB.

- updatexxx() } changes data in the Resultset buffer.
- updateRow() — pushes the changes to DB.

* JDBC Transaction.

- Transaction is a group of SQL operations executes as a single logical unit of work.
- All Operations must either
  - Success together (commit), or
  - Fail together (rollback).

con.setAutoCommit(false);

- Disable JDBC default Auto-commit mode.
- Setting it to false allow manual transaction control

con.commit() ← con.rollback() ← call only when SQL query execute successfully

autocommit = true; ← make it true to change JDBC to its defaut behavious.

* SQL Injection (SQLi)

- It is security vulnerability in which an attacker passes malicious SQL code or special symbol (such as --) through application input.
- This input becomes part of SQL query & alters its intended behaviour.

-- in SQL Injection.

- In SQL -- is comment operator.
- Everything after -- is ignored by the SQL Engine.
- Attackers used it to bypass conditions especially authentication checks.

- Why Statement is Vulnerable to SQL Injection

- Statement build SQL queries using string concatenation.
- Used input becomes part of the SQL Syntax.

"select ... where uname = '" + uname + "' and password = '" + password + "'"

* SQL Injection Scenarios Demonstrate

① Comment Based Injection.

uname = 'admin1' -- and password = "xyz"

- password condition is commented out.
- Authentication succeeds without correct password.

② Always-True Condition Injection.

uname = 'x' OR 10=10 --

↑ Its always true.

entire table matches → logic bypass.

* Why Prepared Statement Prevents SQL Injection.

- It separates SQL logic from data.

select count(*) from myaccount where uname = ? and password = ?

- SQL structure is fixed at compile time.
- User input treated strictly as data, not executable SQL.

* executeQuery()
- Used to execute DQL (select) SQL queries.
- fetching data from DB.

Working:
- JVM sends the SQL query to DB Engine.
- DBE executes the query & fetches records.
- That data returned to Java appl^n as ResultSet object.
- Since DB rows are represented in heap memory as a ResultSet.

public ResultSet executeQuery(String sql)
↑ returns Resultset always

* executeUpdate()
- Used to execute DML queries such as.
- INSERT
- UPDATE
- DELETE
- DDL also (CREATE, DROP etc).

Working:
- DBE modifies DB records & Return the count of affected rows.
- This count is returned to Java Appl^.

public int executeUpdate(String sql);

*execute()
- can execute both selection & updation SQL queries (DQL & DML)
- Used when the SQL type is not known in advance or multiple results are expected.

Working:
- JVM send the SQL query to DBE.
- DBE executes the query.
- Based on query type:
  if query is DQL it returns true.
  - Resultset is created on heap.
  - executes return true.

else
for DML
- Update count is generated.
- execute() return false.

public boolean execute(String sql);

Handling the result of execute().
if(executes() return true).
  getResultSet() ← to retrive data

else.
  getUpdateCount() ← to retrive number of affected rows.

* Connected View of Data:
- A connected environment means
  - The DB connection must remain open while traversing the data.
- Resultset works in a connected mode
- If connection is closed the cursor operation get fail & traversing is not possible.

while Iterating through Resultset, the DB connection must be alive.

*Disconnected Environment:
- connection required only at the time of data retrival.
- After fetching data, connection can be closed.
- Data can still be traversed & processed.

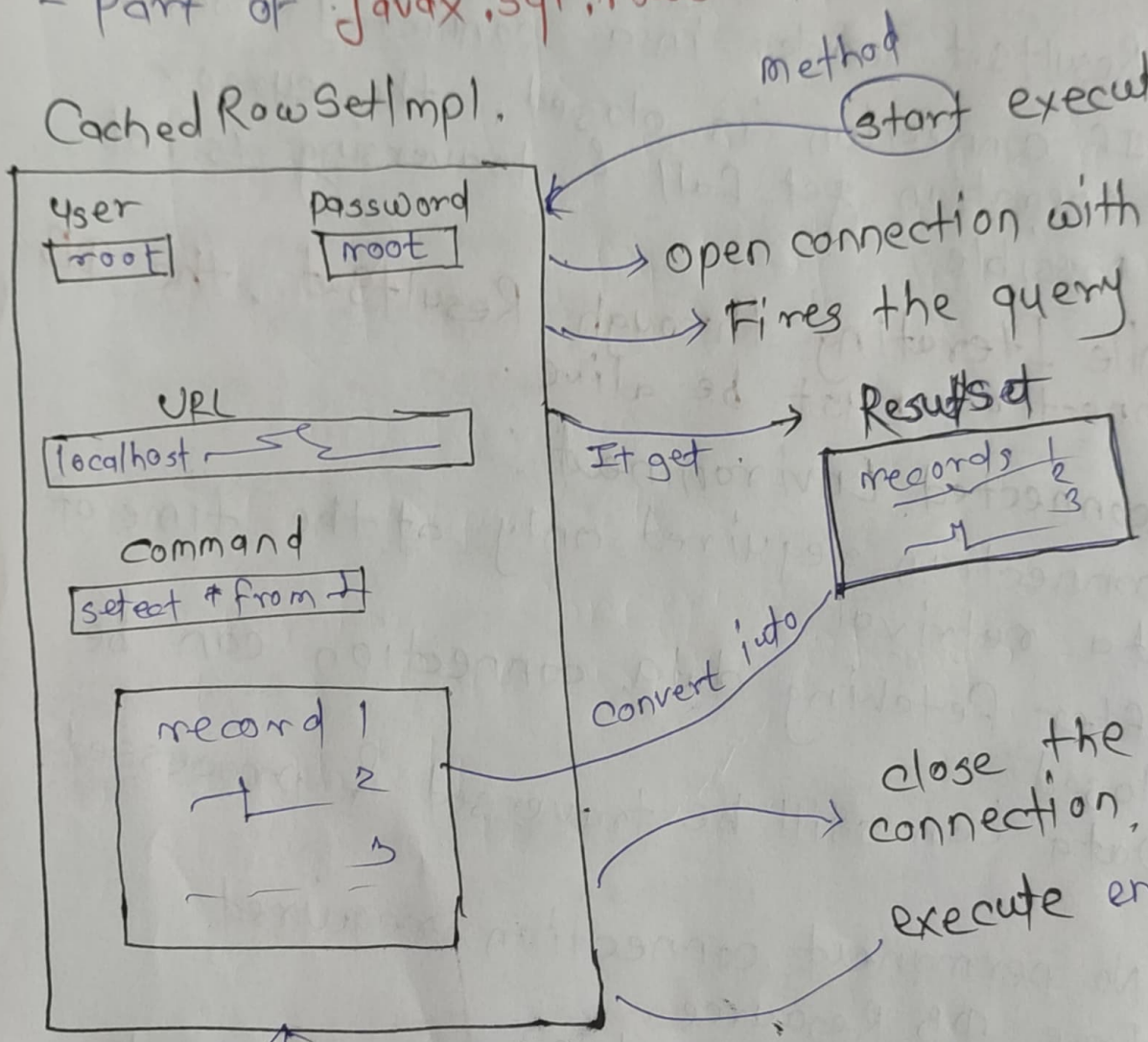
Adv:
- No permanent connection required.
- Free DB Resource.
- Highly benificial when.
  - Many users are accessing Data.
  - Large datasets are involved.
- Reduce load.

Resultset is not Serializable.

CacheRowSetImpl → Serializable.
- Disconnected data can be send over Network.

* CachedRowSet (Disconnected ResultSet)

- It is disconnected, serializable container for tabular data
- Internally stores data in memory
- Part of javax.sql.rowset package.



Application can reads from CachedRowSetImpl even though Connection is closed

holds local copy of result.

CacheRowSet crs = RowSetProvider.newFactory().createCachedRowSet();

class used to create RowSet Implementation.

* Externalizing DB Configuration.

Configuration Values.

```
url = jdbc:mysql://localhost:3306/mydb
user = root
password = root
```

- These values are stored in a properties file.
  - This file must be available on the classpath.
- Avoid hard-coding DB credentials
- Improve maintainability & security.

ResourceBundle:

```
ResourceBundle rb = ResourceBundle.getBundle("myproperty");
```

- utility class in java
- Used to read key-value pair from properties file
- loads properties file automatically.

Read values using keys

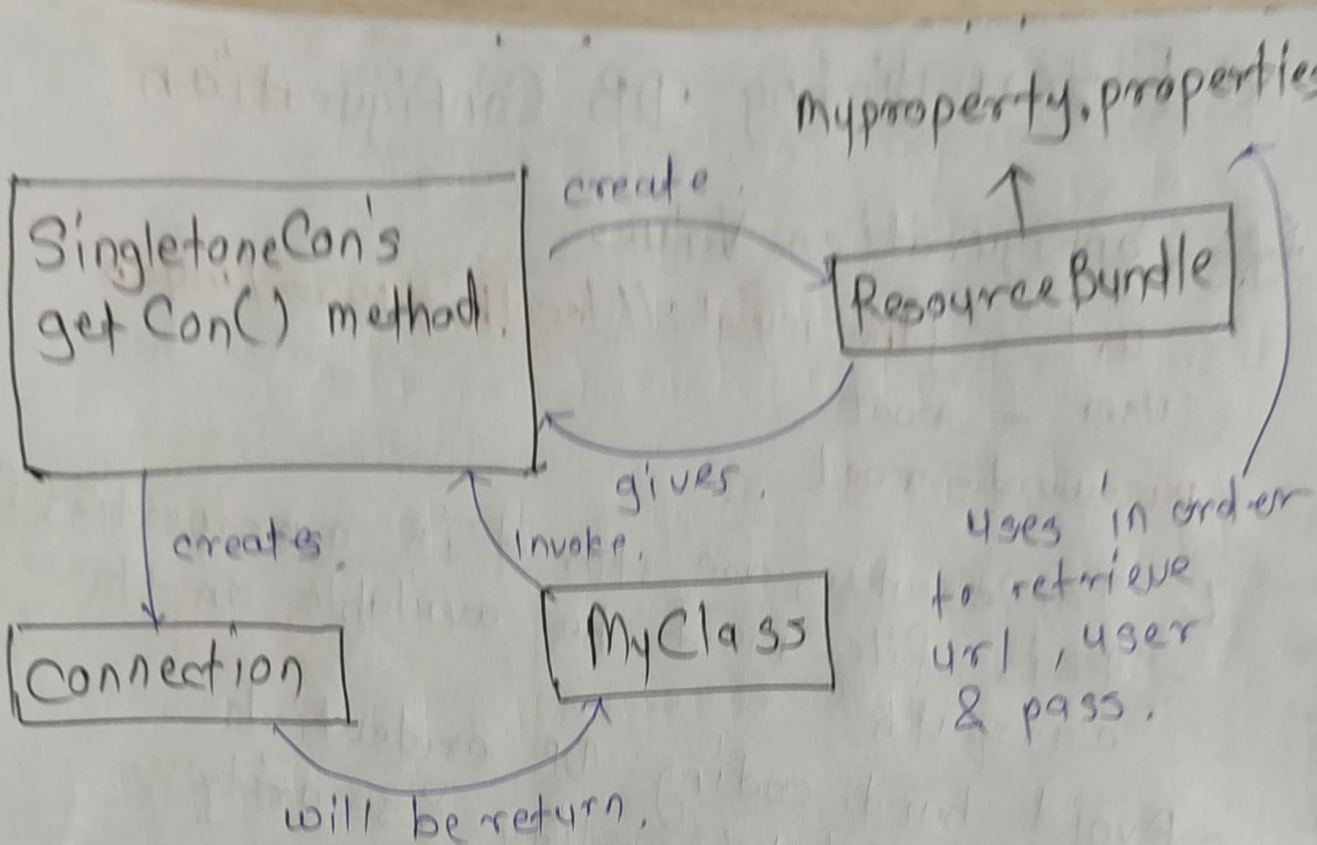
```
url = rb.getString("url");
```

↑ keys are always a strings.

- keep configuration from source code.
- ConnectionProvider class acts as a centralized DB connection factory.
- encapsulate all connection logic in one place.

getCon() ← static method which is reusable to obtain DB connection.

- It ensure Loose coupling & Clean Appl° code.



Why this Approach:

1. Seperation Concerns.
2. Easy Configuration Changes
3. Reusability
4. Cleaner Code.

16/12/2025

(1) Request Responce Model (Static Content)

- The browser sends an HTTP request to the Web Server.
- The server sends responce.
- Responce is static.
- All clients receive the same content.
- No server-side processing logic is involved.

(2) Dynamic Content. (Request - Process - Responce)

- Client sends request.
- Request is processed by server side software component (Servlet, JSP, ASP, ASPt, CGI).
- Server generates responce dynamically based on
  - Request data
  - Business logic
  - User Specific input.
- Diffirent client may receive different responses for the same URL.

* Servelet: ← object is created one per JVM in clustered environment
- It is server-side Java componeut used to generate dynamic web conteut.
- It runs inside Web Coutainer (Servlet Coutainer) such as Apache Tomcat.
- It process clieut requests & generates responses using Request-Response model.
- They mainly handle HTTP requensts in web appl^n.

* Servlet vs CGI

CGI (Common Gateway Iuterface)
- New process is created for each clieut requests.
- Heavyweight & resource-iutensive.
- Low performance.
- Poor scalability

Servlet:
- A New thread is created for each clieut request.
- Run inside a Web Coutainer.
- Lightweight & efficieut.
- High Performance & better scalability.
- Efficieut memory & resource utilization (uses thread pool).

* Servlet API:

- Provided by Java to build web appl^ns
- Defines classes & interfaces required to create servlets.
- Main packages:

javax.servlet —Now→ jakarta.servlet
Generic servlet features.

javax.servlet.http —Now→ jakarta.servlet.http
HTTP specific features.

* Hejrarchy.

Servlet (Interface)

↓ doesn't define http specific request.

GenericServlet (Abstract class)

- protocol Independent
- can handle any request
- Rerely used in web appl^n
- Doesn't provide HTTP-specific methods.

↓ complete all the implementation hence it is abstract.

HttpServlet (Abstract class).

- Design specifically for HTTP protocol
- Provides HTTP-specific methods such as
  - doGet()
  - doPost()
- Most commonly used servlet class in real world appl^n.

* Creating a Servlet:

- Extend HttpServlet class
- Override required methods such as : doGet(), doPost().
- Lifecycle methods if needed.

* Servlet Life Cycle:

- Since servlet runs inside a Web Container, it follows a defined life cycle.

① init() - call only once when servlet is created.
- Used for initialization activities.
- creating DB connections
- Loading configuration data.
- Looking up EJB or remote obj.

you have to write class.forName & Driver.getConnection.

Interface

② service(HttpServletRequest, HttpServletResponse)
- Called for every client request.
- Acts as a dispatcher, method..
- Internally calls:
  - doGet() ← for GET Request
  - doPost() ← for POST Request.

highly recommended not to override service method.

Standalone Vs Web App.
- Standalone ↑ executes locally on client machine without a browser.
- Web App ↑ Accessed through a web browser & runs on a server.

Web Server Vs Appl" Server
- Web Server ↑ serves static resources & handle HTTP requests.
- Appl" Server ↑ Executes business logic & generates dynamic responce.

③ doGet(HttpServletRequest, HttpServletResponse)
- Handle HTTP Get Request.
- Used to retrive data from Server.
- Data is visible in the URL.

④ doPost(HttpServletRequest, HttpServletResponse)
- Handle HTTP POST requests.
- Used to submit data securely.
- Data is not visible in the URL.

⑤ destroy()
- called only once before the serlet is destroyed.
- Execute when:
  - The server is shut down
  - The application is undeployed.
- Used for cleanup operations:
  - closing DB connections
  - Releasing system resources.

jakarta.servlet.http.

ServletResponce (class) → HttpServletResponce. (Interfaces)

ServletRequest → HttpServletRequest

loose coupling, Container Independence.

* Java Enable Web Server.

- Contains a Web container only.
  - manage lifecycle of servlet & JSP.
- Can serve
  - Static resources (HTML)
  - Dynamic web components. (servlet, JSP).
- Not support for Enterprise business components (EJB).

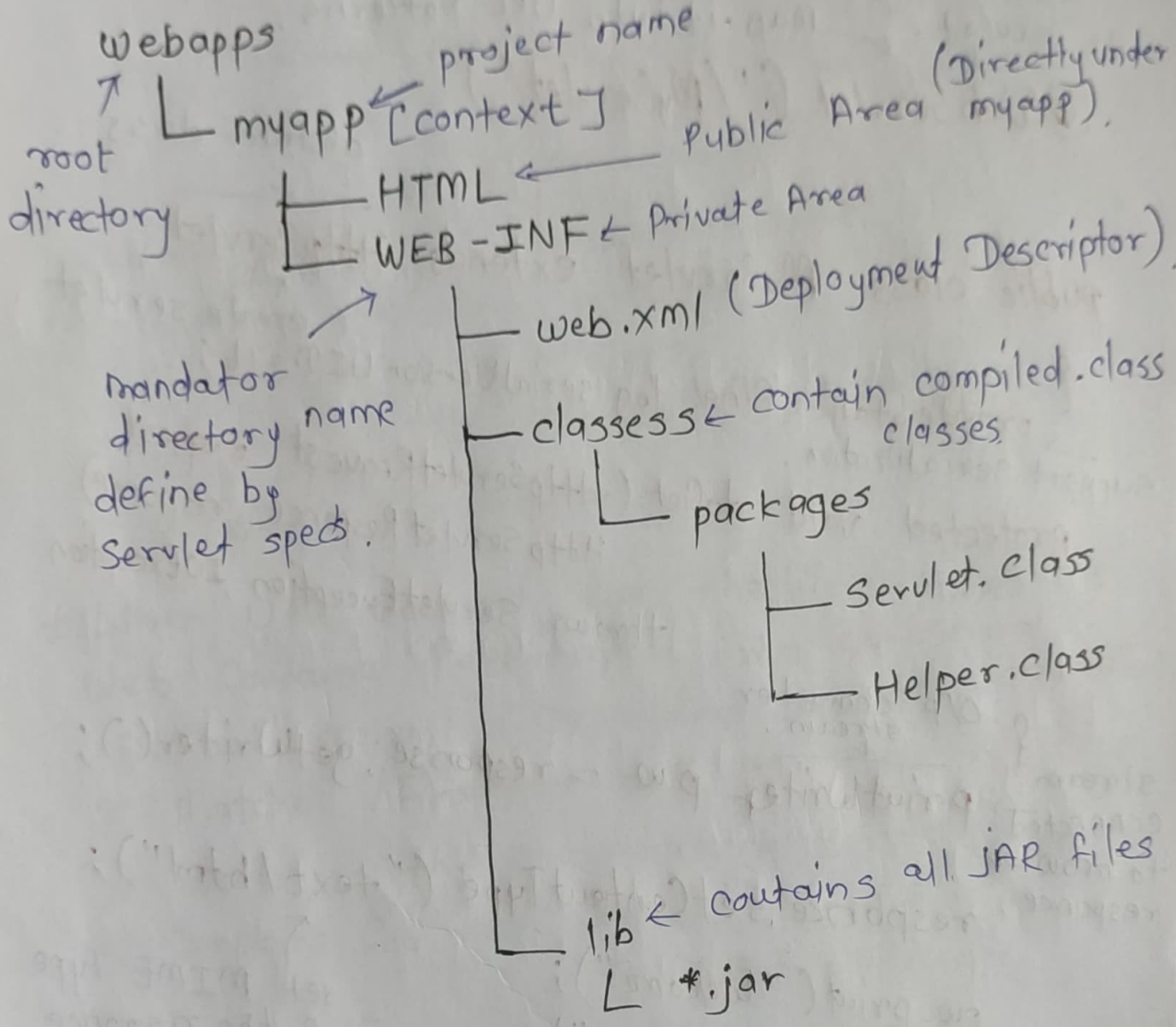
eg: Apache Tomcat, Jetty.

* Java Enables Appl^n Server.

- Contain both Web Container & EJB Container.
  - manages EJB components.
- Provide enterprise services such as.
  - Transactions
  - security
  - Scalability
  - Distributed processing

eg: JBoss / WildFly, WebLogic, WebSphere.

* Java Web Appl^n Structural diagram.



* MyServlet : — before this entry was made in web.xml.

declare class as servlet. ← map it into URL patterns.

@WebServlet ("/MyServlet")

public class MyServlet extends HttpServlet (abstract class) (create servlet)

{

private static final long serialVersionUID = 1L; → support serialization.

protected void doGet (HttpServletRequest request,
HttpServletResponce responce)
throws ServletException, IOException

{ O/P character stream.

stream connected to browser responce. PrintWriter pw = responce.getWriter();

responce.setContentType ("text/html"); ← set MIME type of the responce. - Inform browser that the content is HTML.

pw.print ("Welcome");
pw.print ("
");
pw.print ("All the best");

}

}

MIME type.
Specifies the format & nature of data send from server to client.
Multipurpose Internet Mail extensions

* Servlet Execution:

http://localhost:8080/dac/FirstServ

- http://localhost:8080/ → server & port.
- dac → web appl" (context name)
- FirstServ → URL pattern mapped to a servlet.

- when you change in servlet code.
- following steps are required.
  1. Compile servlet class.
  2. Save / update web.xml. (if mapping are modified).
  3. Refresh the browser.

* When URL accessed for the first time, the Web Container Performs: (From @WebServlet annotation / Deployment Descriptor)
  1. Read web.xml (Deployment Descriptor).
  2. Matches /FirstServ with url-pattern.
  3. Identifies the corresponding servlet class.
  4. Serches for the servlet.class file in WEB-INF/classes
  5. Load the servlet class into memory.
  6. Create Servlet object using the public no_args constructor.
  7. Invoke the init() method (Execute only once).
  8. Allocate or retrieves a thread from thread pool.
  9. Creates HttpServletRequest & Responce objects.
  10. Calls the service() method.
      - It determine the request type
      - doGet(), doPost().

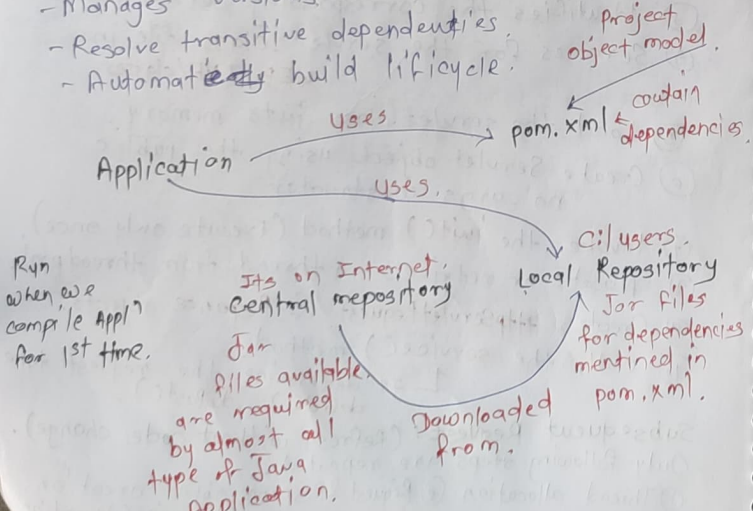
Subsequent Request: (Refresh without code change).
Only following steps are repeated.
1. Thread allocation 2. Request & Responce object creation.
3. Service() execution

* Maven: - project management tool based on POM (project object model.)

Why Maven:

- In traditional Java project:
  - You manually download JAR files
  - You stored libraries inside the project (In lib folder).
  - Handling dependent libraries of libraries is complex.

- What Maven Solves:
  - Maven is a build & dependency management tool that:
  - Automatically downloads required libraries.
  - Manages versions
  - Resolve transitive dependencies.
  - Automatically build lifecycle.

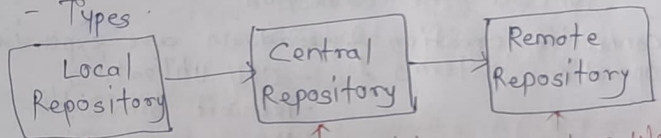


* POM: Project Object Model:

- Fundamental unit of work in Maven.
- It is XML file.
- Always resides in base directory as pom.xml
- contains information about the project & various configuration details.

* Maven Repository:

- Directory of packaged JAR file with pom.xml.
- Maven searches for dependencies in repo.
- Types:



- located in local system.
- created by Maven project when we run maven command.

By default location.
User\Home\m2
↓
Can change by changing settings.xml

- located on Web
- Created by Apache Maven community

- located on Web
- most of libraries can be missing in central repo.
- Its not local Repo & accessed over network (usually via HTTP/ HTTPS)
- called Remote because hosted outside developer machine & accessed remotely over N/w.

* Connection Pooling :

- It is managed cache of pre-established DB connections that can be reused by appl^n to service DB request efficiently.

- Problem Without Pooling
  - In typical DB interaction (Oracle, MySQL)
    - An appl^n
      1. Open a DB connection.
      2. Execute a query or update.
      3. Close the connection
  - connection creation & teardown are expensive operation in terms of
    - CPU utilization
    - Network bandwidth
    - Memory consumption.

- Solution : Connection Pooling
  - A fix configurable number of DB connections are created in advance.
  - Appl^n workFlow
    1. Request (borrow) a connection from pool.
    2. Execute required SQL operations
    3. Return the connection to the pool (instead of closing it).

* Benefits

- Improve performance.
- Reduce resource overhead.
- Better scalability under concurrent load.
- Centralize management of db con.

* Implementation Approach

- Custom-built connection pool (manual).
- Framework-based pooling solution.
  - Apache DBCP
  - HikariCP
  - C3PO
  - Appl^n server-managed pools (Tomcat, JBoss, etc.).

* Connection Pool leak :

- Occurred when the borrowed connection is not returned to the pool.
- Couses :
  - Missing close / return logic
  - Exceptions without proper cleanup.
- Consequences :
  - Gradual exhaustion of available connections
  - Appl^n failures

- Always release connection in finally block or use ARM.

* Without Connection Pooling

DriverManager.getConnection()
↑ create new connection from scratch.

con.close()
↑ close & destroy the physical connection.

* With Connection Pooling (DataSoure)

DataSoure ← interface from javax.sql package.

DataSource ds;
ds.getConneection() ← retrieves an existing logical connection from the pool.

con.close()
↑ return the connection back to the pool for reuse.

* Naming Service
- It maintains a collection of bindings where each binding map a name to an object.
- client interact with it to locate object by name rather than by physical or implementation-specific details.
- It bind a name to an object.
- Look up an object using its name

* Common Naming Service.

(a) COS (Common Object Service)
- Part of CORBA (Common Object Request Broker Architecture)
- store & resolves CORBA object reference.
- Used primarily in distributed object system based on CORBA.

(b) DNS (Domain Name System)
- A distributed directory service
- Maps hostnames. (www.example.com) to IP addresses.
- Fundamental to internet communication.

* Without Connection Pooling

DriverManager.getConnection()
↖ create new connection from scratch.

con.close()
↖ close & destroy the physical connection.

* With Connection Pooling (DataSoure)

DataSoure ← iuterface from javax.s
pack ↑

DataSource ds;=

ds. getConneetion() ← retri
existin
connecti
pool.

con. close(),
↖ return the connecti
to the pool for reus

* Without Connection Pooling

DriverManager.getConnection()
↖ create new connection from scratch.

con.close()
↖ close & destroy the physical connection.

* With Connection Pooling (DataSoure).

DataSoure ← interface from javax.sql package.

DataSource ds;=

ds.getConneection() ← retrieves an existing logical connection from the pool.

con.close()
↖ return the connection back to the pool for reuse.

* Naming Service:

- It maintains a collection of bindings where each binding map a name to an object.
- client interact with it to locate object by name rather than by physical or implementation-specific details
- It bind a name to an object.
- Look up an object using its name

* Common Naming Service.

(a) COS (Common Object Service)
- Part of CORBA (Common Object Request Broker Architecture).
- Store & resolves CORBA object reference.
- Used primarily in distributed object system based on CORBA.

(b) DNS (Domain Name System).
- A distributed directory service
- Maps hostnames. (www.example.com) to IP addresses.
- Fundamental to Internet communication.

(c) LDAP (Lightweight Directory Access Protocol)

- Client-server protocol for accessing & managing directory service.
- Commonly used for authentication, authorization.

(d) NIS / NIS+ (Network Information System)

- Developed by Sun Microsystem.
- Provide centralized access to system configuration data.

* JNDI

Java Naming & Directory Interface.

- It is a JAVA API that provides a uniform interface to access various naming & directory services (existing & emerging).
- Java appⁿ can access different directory service without knowing their internal implementation.

* JNDI API Core Components.

Context : An interface representing a naming context.

InitialContext : default Implementation used to begin naming operations.

Imp method :

void bind (String name, Object obj)
Bind name to an obj.
must not already exist.

Object lookup (String name)
retrieves the obj bound with this name.

* How JNDI Works.

Namespace : java:comp:env.

① DataSource is configure on Applⁿ Server with a logical name jdbc/mypool

② When server starts, JNDI service stores a reference to the Datasource under java:comp/env/jdbc/mypool.

③ The applⁿ performs JNDI lookup to obtain the Datasource.

④ DB connection are aquired from DS without hardcoding DB.
loose coupling & Centralized & portable resource management.

* Types of Parameters:

① Request Parameters: ← automatically set in the request.

Scope - Single HTTP request.
- Valid only during one request-response.
- Automatically set by client (HTML form, query string, URL).

- Used to pass user input or request specific data.
- Automatically populated by the web container.
- Each request has its own set of request parameters.

API: HttpServletRequest

```
String uname = request.getParameter("username");
```

② Init (or Config) Parameters ← log file ↑ keep track of all request

Scope - One specific servlet.
- From servlet initialization to destruction.
- Shared by all request handled by that servlet.
- Defined explicitly in DD (web.xml) or via annotations. API: ~~ServletConfig~~

need to explicitly set in DD. ✗

- Used for servlet-specific configuration.
- One ServletConfig object is created per servlet. (API)
- Same value is available across all requests for that servlet.

(Entries in web.xml eg. (DB connection))

```
String driver = getServletConfig().getInitParameter("driver");
```

③ Context Parameters:

Scope: Entire web application (context)
- From appln startup to shutdown.
- Shared by all servlets, filters & listeners in that appln.
- Defined explicitly in DD. (web.xml)

- Used for ~~all~~ Appln wide configuration.
- Only one ServletObject (API) object per appln.
- Ideal for common resources like DB details, log paths, etc.

```
String dbUrl = getServletContext().getInitParameter("dburl");
```

* Init Method Info.

abstract class.

① Their are two init Methods in HttpServlet.

```
public void init()                          ← Interface in
public void init(ServletConfig config)
```

② Before invoking init method the web Container
- Create an implementation object of ServletConfig.
- Reads all init parameters defined for that servlet in DD / via annotations.
- Store these parameters inside object of

③ Container Invokes init(ServletConfig config).
- It passes the ServletConfig implementation object in previous step.

④ HttpServlet provides a concreat implementation of above method which always invoke init() method.

Internally call two

public void init (ServletConfig config);

public void init();

Inherit both (Not redefine).

Defined in GenericServlet ← abstract class

↓

HttpServlet ← abstract class

↓

ServletConfig ← Interface.

concrete methods.

- So in case of inheriting these methods we can directly inherit them but
  - Init() ← can inherit directly doesn't affect on anything.
- But in case of init(ServletConfig config);

Technically possible, but not recommended.

- Because this method will create the implementation object of ServletConfig so after method ends the object will get destroyed
- So we have to store that object either or pass it to the parent method which will store it for further use.

```
ServletConfig myconfig;
Public void init(ServletConfig config) {
    this.myconfig = config;
    super.init(config)
}
```

either store it

OR

Pass it to parent method.

* Redirect & Forward.

- When single component (Servlet / JSP) is insufficient to process the client request, the processing responsibility is passed to another component.
- This delegation is typically achieved using Request Dispaching.

(a) Redirect (sendRedirect()).

- Client send request to first component.
- that component recommends client to go for another servlet & send redirect responce to client.
- Based on this client issue new HTTP request to another component.
- That component will process request & generate the responce. (It is visible to client which component providing the responce.

- Involve seperate Request & Responce.
- Browser URL changes to redirected resource
- client fully aware about which resource is produced.
- Can redirect to
  - Another Servlet in same Appl^n
  - A resource in diff web Appl^n
  - An external website.
  - It is slower than forward.

Request scope data is not preserve.

API → responce.sendRedirect("AnotherServ");

(b) Forward

- client send request to first web component.
- It partially process request.
- Request forwarded internally to another component.
- The second component completes processing & generates the response.
- The responce is sent back to the client as if it came from the first component.

→ Involve only one HTTP request & Responce.
- The browser URL remain unchange.
- Client believes the first servlet generated the responce.
- Operate strictly within the same web Container.
- Faster than Redirect.
- Request scope data is preserved
- Preferred for internal request deligation. (like MVC)

API: Interface in servlet

RequestDispatcher rd = request.getRequestDispatcher("servi");
rd.forward(request, response);

HttpServlet
Factory method in
path (servlet name) passed to the method
method will map this path & create the object of it.
defines contract for forward & include.
rd = context.getRequestDispatcher("/firstservi");
It will point to that container.
absolute path of root.

(c) Include:

- Method of Request Dispatcher

Used to invoke another web resource & include its output in the current response without terminating the execution of the calling servlet or JSP.

- rd.include(request, responce);
  - Include temporarily transfers control to target servlet.
  - target process the request.
  - generate output in same responce.
  - control returns to calling servlet after inclusion.

- Include returns the control & forward not.
- Responce is included in main servlet by include but in case of forward it sends its seperate responce.
- O/P - include() Appende
  - forward() Replaced.

* HTTP as Stateless Protocal:

- By default HTTP is stateless.
- It doesn't maintain any information about client between two request.
- Each request response cycle is treated independently.
- Server process respons & send to the client & container forgots the client.

* Need for Making HTTP Stateful.

- Typical scenarios requiring statefullness:
  - User authentication & subsequent activity tracking.
  - shopping cart.
  - Multi page forms.

* State Management Techniques in Java.

(a) Hidden Fields.

- Hidden form fields store data in HTML.
- Write in Server.
- The data is preserve in server when form is submitted.
- work only with form.

(2) Custom Cookies.

- Small piece of text based information store on client browser.
- Used to maintain state across multiple HTTP request.
- It can be store only textual info (string key-value)
- Cannot store java object directly.
- They depend on client browser. (Acceptance).
- If user disables coockies they are not save.

flow - ① Request from client.
② server create coockies & send back to with responce.
③ client accept or reject it.
↑ store on browser.
Subsequent req: ① server retrieve coolie from request.

Imp Attributes:

name - key	path - URL path
value - data	domain - domain to which
maxAge - expiry time.	secure - cookie sutouly over HTTPS.
	Httponly - prevent access from JS.

```
cookie[] cookielist = request.getCookies();
response.addCookie(c)
```
↑ add coockie in response.

⑤ Http Session.

- HttpSession interface from javax.servlet.http.
- Server side state management mechanism.
- Store client data on server & maintains user session across multiple request.

Obtain a Session.

Create or Retrieving a Session.

```
HttpSession session = request.getSession();
```

- Both are same. (or `getSession(true);`)
- If session exists, return the existing session.
- If no session exist: It will create new session.

Check whether Session is new or Existing

`session.isNew()` → true: newly created
false: existing.

`session.setAttribute(String name, Object value);`

- Store data in session.
- key value pair. (key must be unique).
- value can be java object.

`session.getAttribute(String name);`

- If value is present of given key returns it or null.

`HttpSession session = request.getSession(false);`

- If session exist it returns the session or not exist then return null.

* Session Flow:

- Container creates a unique object of session. JSESSIONID (cookie which contain session obj) executes in configuration. It sends to the client as cookie (or URL Rewriting)
- to identify the session.
- Session data resides on the server, not on the client.
- default session timeout is 30 minutes (can override xml timeout from java in seconds) write in DD.
  - `setMaxInactiveInterval()`
- Appln can explicitly terminate the session using `session.invalidate();` ← remove all session Attributes, Invalidate the session ID, mark session as destroyed immediately.
  - further attempt to use this session will throw IllegalStateException.
- Server get sessionId from request & try to match it with session object available on server. If match return session else null.

* What if client reject the Cookie Containing Session ID.

- By default container send the session ID through cookie named JSESSIONID
  - If the client reject cookie.
    - the session id not be sent back to the server in subsequent request.
    - As result container cannot identify the client & session tracking breaks.

④ * Alternative Mechanism: URL Rewriting

- This technique appends the session ID to every URL that user clicks.

Methods of URL Rewriting:

server side navigation <a href>

`response.encodeURL(String url)` ← for normal URL

client side redireation

`response.encodeRedirectURL(String url)`

if cookies enable /servl. — if disabled /url;jsessionid=xxx... — for URL used during sendRedirect();

- These methods ensure that URL rewriting is applied only when cookies are disabled.

Example:

```
response.encodeURL("HomeServ");
```

It produce URL like.

HomeServ;jsessionid=AF23H7J5A927...

- When cookies are disable, the method automatically appends the session ID.
- All subsequent request contain that session ID in URL.

Servlet Filters:

ensure that they can be added or remove — executes in configuration order.

- They are pluggable component that can be add/remove without modifying servlet code.
- They are configure through DD or annotations @WebFilter
- Improve maintainability because new requirement it can be implement without altering ~~high modularity~~ servlet logic.
- cross cutting concerns.

Types:

Request Filter: execute before request reaches the ~~target~~ servlet.
- Commonly use for authentication, authorization, request logging.

Response Filter: execute after servlet generate response but befor response goes to client.
- Use for: responce compression, modification, adding header.

Creating Filter Filter.

① Create Java class implements Filter interface.

① `public void init(FilterConfig config)` ← invoke by container once at first filter initialization. - used to read initialize parameters.

② `public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)`
- method where filter logic resides.
- Developer perform preprocessing before calling `chain.doFilter()` & post processing after it.

③ `public void destroy()`
↑ Invoked when the filter is being taken out of service.

Execution Flow.

① Filterchain ← represents the sequence of filter for particular request.

② Container invokes them in configured order.

③ last filter in filterchain target the servlet.

doFilter() ↘ inside it.

chain.doFilter(request, response)

↗ If not called it doesnt proceed further.

↑ can pass control to next filter or servlet (if it is last filter itself)

3 Ways filters are like Servlet.

① Container knows their API.

② Container manages thir lifecycle.

③ Declared in DD or Annotations.

It should be migrated.

* Servlet Instance in Distributed App. (for load balancing)

Session will be migrate using Socket Programming

- Each JVM create one servlet instance for each servlet.
- In distributed system (cluster) each server/JVM has its own instance to handle request.
- That instance are not shared between JVM.
- multiple request handled by multiple threads on same servlet instance.
- load balancers distribute request across server so each server handles request using its own instance.

In order to older version to make the servlet synchronize we have to implements. SingleThreadModel.

* Thread Safety in Servlet:
  - we need to avoid.

* Common error page
  ① make html page
  ② add it into DD

```
<error-page>
<exception-type> java.lang.Exception </exception-type>
        <location> /Error.html </location>
<error-page>
```

* How to create a project & deploy it on any web browser or Application Server.
  - We have to create War File.
    ① War - Web Archive.
    JAR: Java Archive
    EAR: Enterprise Archive.

* Load on Startup Concept (loaded by the container)
  - By default a servlet is stared only when the first request is received for that servlet.
  - When servlet is marked as load on startup it loads during appln startup. Executes it init. keep it ready in memory.

@WebServlet(value = "/InxServlet", loadOnStartup = 1) ← annotations

DD → <load-on-startup>1</load-on-startup> sequence order

* Difference between

Parameters

- Always String
- Provide externally cannot created by servlet code
- Use for configuration & client input.

① request (field/query string)

② Init/Config (In DD/annotation)

③ Context (In DD only).

Attribute

- Can store any java obj.
- created programatically
- purpose is to data sharing across component.

Use for forwarding request (live only during request) till request gets complete. can be

① request — key, obj
request.setAttribut(key, value);

② session ← persist as long as session is alive.
Session.setAttribute(key, value);

③ context
context.setAttribute(key, value);
used to share global object like caches, hit counters.

* JSP: Java Server Pages:

- Server side presentation - layer tech used to build dynamic webcontent in java.
- Middle tier similar to servlet.
- Typically use as a View of MVC.
- Allow UI development by allowing HTML & JAVA - based dynamic elements in the same.
- It considered as advance version of Servlets.
- Although JSP & servlet both generate dynamic content.

JSP : View layer ← primarily handle rendering not processing.
Servlet: Controller layer ← form the entry point for request in MVC.

Every JSP is converted to Servlet by the container before execution.

* Working flow.

The parent class of this servlet is HttpJspBase

① JSP translated into java Servlet (translation phase) ⟹ ② that servlet compiled into .class file (webcontainer)

③ class is loaded by containers class loader

④ Container instantiates it using no-arg constructor

init() ⑤ jspInit() executed once.

⑥ Thread is allocated from the thread pool. ⟹ ⑦ Req, Res objects are created.

service ⑧ _jspService() executes to handle request.

Subsequent Request. (_jspDestroy())
① Container used already created obj. ←
② Thread allocated ⑤ Request, Res. obj
③ _jspService Run ④ If file is modified starts from ①

* Implicit Objects in JSP.

- Container automatically ~~creates~~ provide these object for page execution.

Object (Type).

- request (HttpServletRequest) ← request, parameters, headers
- responce (HttpServletResponse) ← form & send response.
- session (Http Session) ← client session management
- application (ServletContext) ← Application level shared data.
- config (ServletConfig) ← JSP initialization parameters.
- out (JspWriter) ← O/p stream
- pageContext (PageContext) ← page attributes.
- page (Object, typically this) ← reference of JSP page
- exception (Throwable) ← available only in error pages.

* Component of JSP

while translation this comment can go in servlet
<!-- HTML comment -->
<%-- JSP comment --%>

Types:

1. Static Content ← HTML, CSS, JS.
2. Dynamic Content: ← Java-based elements executed on the server.

<jsp:param name="" value="" /> — used to pass the parameter.

out is obj of PrintWriter
↳ Implementation of HttpServletResponce.

* Server Processed Elements in JSP.

* ① JSP Declaration <%! .... %> <%! int a = 10; %>

can write in servlet class
- Used to declare member variables or methods at class level.
- added outside the _jspService() method in generated servlet.

* ② JSP Expression <% = ... %> <%= 3+2 %>

inside SOP.
<%= add(1,2) %>
- Used to output values directly
- Converted to out.println(expression).

* ③ JSP Scriptlets <% ... %>

inside _jspService().
- Contain Java code inserted directly into _jspService()
- can write whole java code (cannot write method);

<% int a = 20;
int b = 30;
~~int sum = a+b;~~ %>

* ④ JSP Directives. <%@ directive attribute = "value" %>

- Used to provide information to JSP translator.

Types

- ① page directive
  - import = "java.util.*"
  - errorPage = "error.jsp"
  - session = "true/false"
  - isErrorPage = "true".

Static include: merged at translation time. include content.

② include directive <%@ include file = "header.jsp" %>

③ taglib directive <%@ taglib uri="..." prefix="..." %>
- Used for JSTL & custome tag.

* ⑤ JSP standard action:

allow us to perform operations without writing Java code, instruct JSP container to perform task.

- Runtime tags provided by JSP specs.
- ① jsp:useBeans ← instantiate or locate existing JavaBeans.
- ② jsp:set/getProperty ← read-write bean properties.
- ③ jsp:include ← runtime include for another resource.
- ④ jsp:forward ← forward request to another resource.

* Static Vs Dynamic include

Directive

<%@ include file="header.jsp" %>

- Processed at translation time
- Fast
- Content merged into one servlet

Standard Action

<jsp:include page="header.jsp" />

- process at request (Runtime) time
- O/P of included (response) is inserted dynamically. (requestDispatcher)

* Business Logic Inside Servlets

- Technically yes. ← we can write business logic inside a servlet.

- However, doing so causes two major drawbacks.
  1. Maintainance Problem
  2. No Reusability ← B logic tied to servlet others cant reuse that logic.

* Business logic Written Seperately.

- When you place Business logic outside Servlets (In seperate class such as service, DAO, pojo.
- Adv: 1. Better maintainance.
  2. Reusability

* Using Business Logic in a Servlet.

1. Logic should be placed in seperate Java Class. also called as JavaBean.

   POJO (Plain Old Java Object)

- Servlet use this logic by creating an object of bean inside doGet().
- It can be reuse with any servlet.

* Using a Business Class Developed by Another Developer.

- The other developer gives you a JAR file.
- Place it into ~~lib~~ WEB-INF/lib.
  - tomcat automatically loads it
  - It can available for entire appl.

* JSP (Attributes).

- In JSP data can be stored using four scopes.
- Each have its specific lifetime.

scope/attribute	object	
page	pageContext	← only within same JSP page
request	request (can differ only in forward, same)	← for one request-response cycle only.
session	session	← for user session until browser close the session or expire.
application	application	← for entire web appl" until server restarts.

① Page Scope.

Object: pageContext.
- Available only in the same JSP page.

Ex: `pageContext.setAttribute("module","java");`
`pageContext.getAttribute("module");`

② Reques Scope

Object: request.
- Attributes is valid only for a single request
- A completely new request loses the data.
- Different request cannot share request Attributes
- Request attribute survive across forward/include because it is the same request object.

③ Session Scope.

Object: session
- Attributes lasts for entire browser session
- Accessible across multiple JSPs/Servlets for same user.
- If browser is closed and reopened.
  - Session lost because a new session is created.

④ Application Scope.

Object. application (ServletContext)
- Shared by all JSPs & all users.
- Remain available until the server is restarted.
- closing browser doesn't affect on it.

* Standard action.

① JSP : UseBean. ← import the java file in JSP.

`<jsp:UseBean id="obj" class="mypack.MyBean" scope="session"/>`

- id = "obj" → object name
- class = "mypack.MyBean" → class name - package.classname
- scope = "session" → Not compulsory - by default page

equivalent to

`SessionContext.setAttribute(ref, new mypack.MyClass());`

compulsory

It is Java

can be change according to scope.

④ JSP:param

- Used to pass parameter to other actions such as include, forward or plugin.

```
<jsp:param name="vname" value="admin" />
```

with include.

```
<jsp:include page="x.jsp">
    <jsp:param name="id" value="101" />
</jsp:include>
```

③ jsp:include

- perform dynamic include at request time.
- include O/P from another resource.

```
<jsp:include page="header.jsp" />
```

- executes at runtime
- Always includes the latest content.
- Different from <%@ include %> which is static.

① jsp:forward

- Forward same request & responce to another resource.

```
<jsp:forward page="next.jsp" />
```

- No response is sent before forward.
- After forwarding, the original JSP stop execution.

⑤ jsp: ~~setAttribute & getAttribute~~ setProperty & getProperty.

```
<jsp:setProperty property="name" name="obj" value="" />
```

property name, object of class

⑤ jsp:setProperty

```
<jsp:setProperty property="name" name="obj" param="name" />
```

obj of class; property name; parameter from form

```
<jsp:setProperty property="*" name="obj" />
```

- can be use for any property
- No need to define the param (parameter name of obj)
- The ~~name~~ per field name of HTML form is must be same as ~~its~~ properties of POJO class.

```
<jsp:setProperty property="list" value="<%= new ArrayList() %>" />
```

- cannot set dynamically
- only can get.

jsp:getProperty.

```
<jsp:getProperty property="age" name="obj" />
```

* EL (Expression Language)

- Feature of JSP allow developer to access stored in various scope
  - page
  - request
  - session
  - application

  & retrieve parameter attributes.
- Simplified Syntax.
- Provide cleaner, more readable alternative to using scriptlets or explicit java code within JSP pages.
- It is null friendly.
- If expression evaluate null
  - It doesn't throw NullPointerException.
- It simply returns null or Empty String depending on context.

Syntax:

${ expression }

Ex. ${user.name}
${param.id}
${sessionScope.cartItems}

<%@ page isELIgnored = "false" %>

* EL Implicit Objects.

- It provides build-in implicit objects.
- They internally stored into MAP. (key - value).
- We can access the data by using objects (using (key) to attributes stored in that Map.

① pageScope
② requestScope
③ sessionScope
④ applicationScope

} map

Each one of them store the data inside their own 'map'. by using these object we can access that data. (using key).

Ex.

${pageScope.msg} ← when we set ang page Attribut the key & value stored in the MAP.
${requestScope.total}
${sessionScope.username}
${applicationScope.version} depends of the attribute type.
${version} ← Its also fine but slowed due to it traverse whole helrarchy

* Parameter based Implicit Object:

param. ← return parameter value as string

paramValues ← return parameter values as an array or list if multiple values exist (e.g. checkbox).

${param.name}
${paramValues.hobbies[0]}
↑ String Array

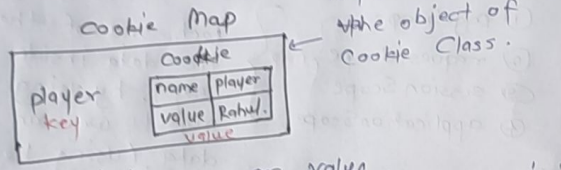
Cookie Access.

cookie ← implicit object of cookie

<%.

Cookie c = new Cookie ("player", "rahul");

name ↓ value ↓



key is representing the object of Cookie Class.

- To access that name or value.

${ cookie.player.name } ← print the name of key inside the Cookie Class obj.

${ cookie.player.value } ← prints the value inside obj.

${ cookie.player } ← it prints the object directly.

Internally EL call the Getter & Setters of Cookie

* Design Patterns:

- They are proven, reusable solutions to commonly occuring problem in S/W design.
- They represents best practices refined through experience rather than finished code

* Gang of Four (GoF)

- Refers to 4 authors - Erich Gamma
  - Richard Helm
  - Ralph Johnson
  - John Vlissides.
- In 1994 they published the seminal book Design Patterns: Element of Reusable OOP s/w

* Core OOP Design principle (GoF).

- The GoF Design pattern grounded in two fundamental principles.

① Program to Interface, not an implementation.
- depend on abstraction rather than concrete classes.
- Enhances flexibility & supports polymorphism

② Favor object composition over inheritance.
- Achieve reusability by composition rather than inheritan.
- Reduce tight coupling & increase Adaptability

* Classification of GoF Design Pattern.

① Creational Patterns ← focus of obj creational mechanism, abstracting the instantiation process.
- Factory Method
- Abstract Factory
- Builder
- Prototype
- Singleton.

② Structural Patterns. ← Deal with obj composition & class relationships to form larger structure.
- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight.
- Proxy.

③ Behavioral Patterns. - Concerned with communication, responsibility & interaction between object.
- Mediator
- Memento
- Interpreter
- Iterator
- Chain of Responsibility
- Command
- State
- Strategy
- Observer
- Template Method
- Visitor.

Ⓐ Creational Pattern:
- Refers to creating obj in s/w design
- Concerned with the way of objects are instantiated or created.

① Singletone.
- Ensures that only one instance of a class is created & provide a global point of acess to that instance during lifetime of appln.

* Typical Scenarios are include:
- Thread Pools
- Caches
- Logging objects.
- Configuration or registery handle.
- Device driver.

* Characteristics:
- Private Constructor.
- static instance variable.
- public static accessor method.

* Basic Lazy initialization (Not thread safe)

```
public static Singletone {
    private static Singletone INSTANCE;
    private Singletone () { }

    public static Singletone getSingletone() {
        if (INSTANCE == null) {
            INSTANCE = new Singletone();
        }
        return INSTANCE;
    }
}
```

- obj is created only when required (lazy
- Suitable for resource intensive obj. initialization)
- Not thread safe.
- When 2 threads execute method simultaneously
  - Both may see the INSTANCE == null.
  - Both create a new obj.
  - Result is violating the pattern.

option ① Soln: Make it synchronize method.
- It simple & guerantees only one instance.
- But we only need singleton synchronization while creating the object at very first time.
- After that synchronization overhead every method call.

option ② : Eager Initialization (Thread-Safe)
- Instance is created when class is loaded.
- JVM guerantees thread safety during class loading

Adv: Simple & No synchronization overhead.

DisAdv: Instance is always created even if not used.

option ③ ~~Double Checked logg~~ Double Checked locking (Recommended)
- Efficient & thread safe

```
public static singletone getSingletone()
{
    if (INSTANCE == null) {          // avoid synchronization after instance creation.
        synchronized (Singletone.class) {   // happen only once
            if (INSTANEE == null) {
                INSTANCE = new Singletone();
            }
        }
    return INSTANCE
}
```

Example of Singletone in JDK:

(1) public class Runtime extends Object
- Only one instance / Java appl^
Runtime r = Runtime.getRuntime();
- Used to interact with JVM Runtime environment.
- Executes System commands.
- Manage memory & processes.

(2) java.awt.Desktop
- Represent the desktop env.

* 3 ways to break Singletone.

(1) Clonning
- clone method can create new obj.
- To avoid dout implement Clonable.
or override clone & throw exception.

(2) Serialization - In serialization while deserilizing
java creates new instance.
- To avoid dout ~~use~~ Implement serializable or
Implement ~~@~~ readResolve() to return the existing instance.
- During deserialization JVM replace the newly created
~~(3) Reflection~~ object with existing instance.

(3) Reflection:
- Allow access of private constructor, enabling
multiple instance creation.
- To avoid throw exception inside constructor if instance
is already created

Enum is the best for Singleton ← In serialization only reference is serialized & while deserialization. It returns existing obj.

* Simple Factory Pattern: (Its just a solution not a design pattern)
- ~~In oop directly~~
- Simple Factory is a design technique that encapsulate object creation logic in a single class, instead of allowing clients to create objects using new.

Proble (before):
- Client code directly creates object using new.
- Same object creation logic Repeated multiple time.
- Any change in concreate classes required changes in all client codes.
- Result tight coupling & high maintainance cost.
- It uses Program to implementation instead of interface. (Violates DP Rules)
- So Solution: Simple factory Pattern.
- Move all object creation logic into a seperate factory class.
- Client request objects from the factory instead of using new.

simple factory centralizes object creation by providing a method that returns instances based on input.

Example:
- UIComponentFactory is the interface which Implemented by several child classes
  eg:- Radio Button
  - Checkbox
  - Dropdown
  - Button etc

- We create one seperate method which provide the object as per demand on runtime maked code loosely coupled & program to interface.

reference of Interface

```
public class GUIApp{
    private UIComponentFactory factory;
    public GUIApp(UIComponentFactory factory){
        this.factory = factory;
    }
    public UIComponent getUIComponent(String type){
        return factory.createUIComponent(type);
    }
}
```

- As soon as we create the object of above class it will ~~create~~ assign the reference to the child as per user input.
- get method return the object of passed input as per demand.

- Simple factory supports only one way of creating object.
- Problem arises when object creation varies by context or platform.

* Need for factory method
- Different platform require different implementation of same product.
- Object creation logic should vary without modifying existing code.
- Instantiation responsibility must be delegate to subclasses.

* Factory Method Pattern.
- Defines an interface for creating an object, but lets subclasses decide which class to instantiate.
- Factory method lets a class defer instantiation to subclasses.

Core Idea.
- Superclass define a factory method.
- Subclass override the factory method.
- Subclasses decide which concrete class to create.

```
abstract class GUIFactory{
    abstract UIComponent createComponent(String type);
}
```

factory method shifts object creation from a single factory to subclasses, enabling ~~in~~flexibility.

* Abstract Factory:
- Provides an interface for creating a family of related or dependent object without specifying their concrete classes.

eg In case of OS (windows, linux, MacOS), each have - Button
- Textfield
- checkbox.

- All must belong to same family.
- In case of factory method when we try to create multiple related products together, factory method is insufficient.
- Abstract factory defines multiple factory methods, one for each product type.
- Each concreate factory represents a platform of family.

* Builder Pattern:
- It constructs complex object step by step.
- Seperate object construction from representation.

~~Problem & Solves~~,
- It holds all the attributes required by main class & provide step-by-step methods to set these attributes.
- After that builder creates a new instance of main class.
- This seperates the construction logic from the main class.
- Instead of creating the object & main class every time.
- Enabling the same method to create different variations of an object.

* Prototype Pattern:
- Use when object creation is expensive, time consuming or complex.
- & If you already have an existing object that is very similar to what you need.
- Instead of creating the new object from scratch you clone the existing object & modify it.
- We keep prototype object (an existing instance).
- Create new object by cloning the prototy
- modify it accordingly.

* Structural Design Pattern:

- It refers to how object & classes are arranged or composed to form larger more complex structures.

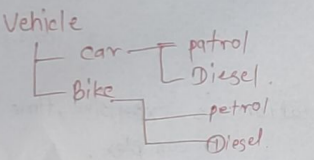
- It focus on
  - Object composition
  - Class organization
  - Relationship between objects
  - Efficient structuring of components.

Examples.

① Bridge Pattern:

- Seperate the abstraction from its implementation so they can evolve independently.

Problem without Bridge.



- In future if i want to add electric engine i need update in full hierarchy.
- This problem is called class explosion because every new dimension creates multiple new subclasses.
- The Reason is we are mixing two independent dimesions inside one hierarchy. Vehicle Type.
  It uses is-a relationship → Engine Type

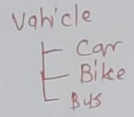
- Bridge pattern seperate this abstraction from its implementation.

This means:

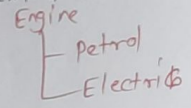
- Extract the second dimension (engine-type) into its own heirarchy.
- Lets the first dimension (vehicle type) (Container) have a engine object (content).
- So instead of - Car is-a petrolCar.
  you use - Car has-a Engine.

After Applying Bridge.

Abstraction Hierarchy



Implementation heierarchy



- The engine reference inside Vehicle acts as a BRIDGE to implementation Hierarchy.